

SMITHIAN FOLD THEORY V3

# Independent Lean 4 Verification of the Complete Smithian Fold Theory Model

A kernel-checked operational root, proof-bearing acceptance gates and fail-closed audit of all current branches

---

Maria Smith

Independent researcher and founder, Ernos Labs

Maria.Smith.Sftoe@gmail.com

Version 1.0.0

2 August 2026

DOI: not assigned - local publication candidate

Paper: CC BY 4.0 - Code: Apache-2.0

Lean 4 whole-model report: PASS

SHA-256 6b6a5a9a9a0b74687a6f97fb3b4fcc7455968e35c82395290837cda60d7501cf

**UNPUBLISHED LOCAL PUBLICATION CANDIDATE**

**MARIA SMITH'S EXPLICIT CONFIRMATION IS REQUIRED BEFORE RELEASE**

# Contents

|                                                                                                                 |    |
|-----------------------------------------------------------------------------------------------------------------|----|
| Contents                                                                                                        | 2  |
| A kernel-checked operational root, proof-bearing acceptance gates and fail-closed audit of all current branches | 3  |
| Abstract                                                                                                        | 3  |
| Headline findings                                                                                               | 3  |
| Current status, evidence language and reader map                                                                | 4  |
| Corpus-wide terminology and evidence key . . . . .                                                              | 4  |
| Three reading levels . . . . .                                                                                  | 5  |
| Editorial change control . . . . .                                                                              | 5  |
| Scope, ownership and evidence constitutions                                                                     | 5  |
| 1. Question and contribution                                                                                    | 5  |
| 2. Protected-boundary design                                                                                    | 5  |
| 3. Formalization of the two-class operational root                                                              | 6  |
| 3.1 Registered grammar . . . . .                                                                                | 6  |
| 3.2 Presentation supplies an occurrence . . . . .                                                               | 6  |
| 3.3 Existence and uniqueness . . . . .                                                                          | 6  |
| 3.4 Scope of uniqueness . . . . .                                                                               | 7  |
| 4. Proof-bearing whole-claim gates                                                                              | 7  |
| 5. Whole-model executable audit                                                                                 | 7  |
| 5.1 Inputs . . . . .                                                                                            | 7  |
| 5.2 Complete result . . . . .                                                                                   | 8  |
| 5.3 Complete claim-inventory boundary . . . . .                                                                 | 8  |
| 6. Preserved fail-closed event and correction                                                                   | 8  |
| 7. Interpretation: how the result bears on SFT                                                                  | 9  |
| 8. Reproduction                                                                                                 | 9  |
| Verification-source identities . . . . .                                                                        | 10 |
| Open science, criticism and admission                                                                           | 10 |
| 9. Falsification and review conditions                                                                          | 10 |
| 10. Limitations and next formalisation frontier                                                                 | 11 |
| 11. Conclusion                                                                                                  | 11 |
| Data and code availability                                                                                      | 11 |
| References                                                                                                      | 11 |
| Suggested citation                                                                                              | 11 |
| Exact current claim-record preservation addendum                                                                | 12 |
| Premise-free operational root theorem . . . . .                                                                 | 12 |

# A kernel-checked operational root, proof-bearing acceptance gates and fail-closed audit of all current branches

**Author:** Maria Smith, independent researcher and founder, Ernos Labs

**Publication authority:** Maria Smith

**Version:** 1.0.0

**Date:** 2 August 2026

**Status:** Final local standalone publication candidate; not approved, deposited or published

**DOI:** Pending a new standalone record after explicit confirmation; no DOI has been assigned or invented

**Paper and documentation licence:** CC BY 4.0

**Verification code licence:** Apache-2.0

***Publication control.** This manuscript and its evidence bundle are prepared*

*locally. No push, upload, DOI creation, release or publication is authorised*

*until Maria Smith confirms the exact candidate files and hashes.*

## Abstract

This paper reports an independent Lean 4 verification layer for the complete current Smithian Fold Theory (SFT) model without modifying the model, its frozen admission engine, its verification authority, any claim package or any historical receipt. The project pins `leanprover/lean4:v4.32.0`. It formalises the registered operational root as an exhaustive two-constructor type, proves constructively that `presentedOccurrence` is the unique survivor, and reports no imported or user-declared axiom for the exported root theorems under Lean's `#print axioms` audit. It separately formalises twelve whole-claim acceptance gates and constructs a proof-bearing certificate only if every gate is true.

The compiled verifier then reads the complete ordered model census and execution manifest as data. It checks 2,777 live claims across 17 branches, 898,902 candidate records, 898,902 decision records and 11,108 controls. It verifies identity, source custody, dependency closure, candidate completeness, decision coverage, exactly one survivor, minimality, named-form uniqueness, controls, empirical boundary, certificate binding and admitted-receipt status for every claim. The final result is `PASS` with 0 issue. The machine report is `generated/lean4_validation/reports/whole_model_validation.json` with identity `sha256:6b6a5a9a9a0b74687a6f97fb3b4fcc7455968e35c82395290837cda60d7501cf`.

The result strengthens SFT's formal-coherence, exhaustiveness, provenance and cross-branch consistency case. It does not convert the remaining 2,776 scientific statements into native Lean propositions, replace empirical tests, or establish superiority to every rival theory. Those boundaries are part of the verified claim discipline rather than exceptions to it. Lawful criticism, falsification and versioned extension remain open, and empirical and publication status remain distinct.

## Headline findings

1. **Unique operational survivor.** Lean 4 proves constructively that `presentedOccurrence` is the unique survivor of the exact registered two-class operational grammar.
2. **No added theorem axiom.** The exported root theorems report no imported or user-declared axiom under `#print axioms`; no `sorry` is present.
3. **Complete current-census audit.** All 2,777 ordered claims receive proof-bearing accepted gate certificates after 898,902 candidates, the same number of decisions and 11,108 controls are checked.
4. **Complete branch surface.** All 17 registered branches reconcile with no omitted census claim and no reported issue.
5. **Independent, read-only architecture.** The Lean layer does not modify or replace the protected engine, verification authority, claim packages or receipts.

6. **Demonstrated fail-closed behaviour.** Six byte-level source mismatches halted the initial run; the expected identities and acceptance logic were not weakened, and the corrected exact-byte state passed.
7. **Bounded scientific meaning.** The result supports formal coherence and custody. It does not independently establish every empirical claim or comparative superiority.

## Current status, evidence language and reader map

| Status field                  | Current position                                                                                                                                                                           |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Paper and completion boundary | methodological or comparison scope; no downstream claim ownership transferred. Completion is dated to the current registered census and remains open to lawful versioned extension.        |
| Formal status                 | The Lean root theorem is kernel checked. Every live model claim has a current admitted receipt and passed the Lean artifact gates. Formal admission does not imply empirical confirmation. |
| Empirical status              | Claim specific. Blind, non-blind, development-observed, holdout, favourable, adverse, null, missing, unavailable, disputed and unresolved distinctions remain exactly as recorded.         |
| Chronology                    | Claim-specific registration, sealing, custody and observation order controls. Later evidence never becomes an earlier prediction without the registered chronology.                        |
| Publication status            | Version 1.0.0 is a final local publication candidate awaiting Maria Smith's approval. It is not deposited or published.                                                                    |
| Ownership                     | This paper retains its declared scientific boundary. Lean verification creates no new branch ownership and no application may select a law.                                                |
| What is not claimed           | Permanent closure of science, universal empirical confirmation, conversion of compatibility into confirmation, or superiority to every rival model.                                        |

## Corpus-wide terminology and evidence key

| Term                                     | Reserved meaning in this paper                                                                              |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Theorem                                  | A formally closed proposition at its declared grammar and dependency boundary.                              |
| Law                                      | An admitted branch relation with its declared carrier, boundary, dependencies and extension rule.           |
| Claim                                    | The registered unit judged by the engine and bound to one immutable receipt.                                |
| Constitution                             | The rules governing admissible objects, derivations, evidence and publication; not an empirical result.     |
| Derivation                               | Generation and elimination of the declared candidate space to its surviving structure.                      |
| Prediction                               | A consequence sealed before the matching target is released under the registered custody protocol.          |
| Observation                              | A source-bound external record; it does not enter candidate generation.                                     |
| Measurement                              | An instrument-, method-, condition- and uncertainty-bound observed value.                                   |
| Reconstruction                           | A separately implemented regeneration or an explicitly identified inference of a retained state or history. |
| Exact numerical correspondence           | Equality or registered interval relation between exact numerical objects at the declared boundary.          |
| Structural correspondence                | A post-derivation relation between forms without a claim of exact numerical prediction.                     |
| Boundary correspondence                  | Agreement restricted to the named interface, limit or ownership boundary.                                   |
| Compatibility                            | Non-adverse but non-discriminating evidence.                                                                |
| Support                                  | Relevant evidence that is not unique confirmation.                                                          |
| Confirmation                             | Used only when the current registered evidence protocol warrants that classification.                       |
| Validation                               | Successful execution of the named formal, computational or empirical test; its kind must be stated.         |
| Adverse result                           | A registered test result that conflicts with the tested claim at its declared boundary.                     |
| Unresolved result                        | Required evidence remains unavailable, incomplete, disputed or insufficiently classified.                   |
| Implementation identity                  | The hash-bound identity of executable material; implementation success is not empirical confirmation.       |
| Formal, empirical and publication status | Three independent classifications; none substitutes for another.                                            |
| Foundational closure                     | Completion of the registered foundation only; not field-wide closure.                                       |
| Field-wide closure                       | Completion of the named frozen or registered dated field census.                                            |
| Current-evidence closure                 | Closure only to the evidence surface presently registered and preserved.                                    |
| Extension openness                       | New questions may enter a later version without rewriting receipts or history.                              |

Proof language is reserved for formal closure; derivation for generated structure; implementation for executable demonstration; prediction for a correctly sealed prospective consequence; observation and measurement for source-bound external records; reconstruction for separately regenerated or inferred states; correspondence for post-derivation relations; compatibility and

support for non-unique evidence; confirmation only where the registered protocol authorises it; adverse for conflict; and unresolved where required evidence remains incomplete.

### Three reading levels

1. **Conceptual paper.** The abstract, headline findings, branch narrative, major derivations, evidence synthesis, limitations and conclusion provide the readable scientific argument.
2. **Scientific audit layer.** Family and claim sections preserve exact status, dependencies, candidate and survivor counts, controls, chronology, sources, corrections, adverse evidence and reconciliation.
3. **Machine archive.** The repository packages preserve complete candidates, decisions, hashes, receipts, executable traces, source snapshots and certificates. They remain authoritative where prose abbreviates display.

### Editorial change control

This successor improves currency, expression, typography and navigation. It does not change scientific meaning, chronology, evidence class, branch ownership, claim status or machine identity. Any conflict between authoritative records must be resolved by explicit supersession or reported to Maria Smith; prose alone cannot manufacture agreement.

## Scope, ownership and evidence constitutions

The verification paper owns the description and reproduction of the Lean layer. It does not take scientific ownership from Foundation, Mathematics, Information Science, Computation, Quantum Computation, Physics, Chemistry, Materials, Biology, Medicine, Consciousness, Earth, Astronomy, Social and Collective Systems, Engineering Translation or Cross-Branch Synthesis.

The mathematical constitution is the explicit inductive root grammar, its decision function, constructive unique-survivor proof and the proof-bearing acceptance structure. The computational constitution is the pinned toolchain and compiled read-only traversal of the exact model artifacts. The empirical constitution remains in the claim packages: source identity, transport, registration, seal, custody, observation or measurement, adverse and unresolved rows and falsification rule. Lean verifies those records and gates as artifacts; it does not become the measuring instrument.

Chronology remains claim specific. The root formalisation was written after the model and therefore validates an existing registered theorem; it is not retroactively classified as the theorem's original prediction. The source-custody halt precedes the final PASS and remains part of the verification history.

## 1. Question and contribution

The verification question is exact: can an independent proof-assistant layer reproduce the registered two-class operational root, prove its unique survivor, and traverse every current SFT branch without changing the model or its admission authority? The answer for the repository state bound by this paper is yes.

The contribution has three parts. First, the root theorem is stated in Lean as a finite inductive grammar whose constructors correspond exactly to the two registered operational challenge classes. Second, the acceptance constitution is represented as a proof-bearing structure. Third, a compiled Lean executable audits the complete machine artifact graph and refuses success if any required gate is absent, malformed or false.

This is validation, not a new derivation route. The SFT engine remains the only component that admits a model claim and writes an authoritative receipt. The Lean layer is read-only with respect to all scientific artifacts. Its report records what it independently reconstructed from those artifacts.

## 2. Protected-boundary design

The verification project lives under `generated/lean4_validation/`. Its local toolchain and build outputs are isolated under `.elan/` and `.lake/` and ignored by version control. The executable receives the repository root and an output-report path. It reads the census, execution manifest, claim packages, certificates, controls, source registrations and receipts. It writes only its report under the Lean project.

Before and after the work, the repository's protected engine seal and verification-authority seal are checked by their existing seal verifiers. No file under `sft/engine/` is imported as editable proof-assistant source, and no authority hash is regenerated to

accommodate the Lean result. This asymmetry is deliberate: Lean may inspect the admitted record, but it may not become a hidden second admission route.

The source binding gate needs the registered Python factories because those factories define how several source manifests are instantiated. A small read-only bridge, `source_binding_probe.py`, imports those registrations, computes their current bindings and returns JSON to the Lean process. It does not call the engine's admission operation and cannot issue a receipt. The Lean verifier treats a failed bridge, malformed response, missing registration or false binding as a failed gate.

## 3. Formalization of the two-class operational root

### 3.1 Registered grammar

The root claim is `SFT-ROOT-THERE-IS-NO-NOTHING`:

*No admissible operational statement, denial, record, proof object or derivational object is nothing: absence presents no counterexample, while every presented counterexample is an occurrence.*

Its registered candidate generator partitions every purported operational counterexample by whether it emits an identifiable presentation at the proof boundary. The exact census contains two forms:

1. `unpresented-absence`: no statement, denial, record, trace or identity is presented, so there is no operational counterexample to admit;
2. `presented-occurrence`: a statement, denial, record, trace or identity is presented, and that presentation is an occurrence rather than nothing.

The Lean type mirrors only that registered partition:

```
inductive ChallengeClass where
| unpresentedAbsence
| presentedOccurrence

def survives : ChallengeClass -> Bool
| .unpresentedAbsence => false
| .presentedOccurrence => true
```

The type is finite by construction and has no hidden third constructor. `challengePartitionComplete` proves by cases that an arbitrary member equals one of the two named constructors.

### 3.2 Presentation supplies an occurrence

The formalisation makes the operational witness explicit. A `Presentation` carries an occurrence identity. `Occurrence.ofPresentation` constructs the occurrence supplied by that presentation, and `OccurrenceOf presentation` is the subtype whose member is exactly that occurrence. `presentationIsOccurrence` proves `Nonempty (OccurrenceOf presentation)` for every presentation by giving the constructor and reflexivity proof.

This theorem does not assert that every imaginable metaphysical account has been encoded. It states the registered operational boundary: once a challenge is presented as a checkable object, the presentation itself supplies an occurrence at that boundary.

### 3.3 Existence and uniqueness

Define

```
def HasUniqueSurvivor : Prop :=
  □ candidate : ChallengeClass,
    survives candidate = true □
  □ other : ChallengeClass,
    survives other = true -> other = candidate
```

Existence is witnessed by `presentedOccurrence`, whose decision reduces definitionally to `true`. For uniqueness, take any surviving candidate and eliminate it by the two constructors. In the `unpresentedAbsence` case, the hypothesis reduces to `false = true`, which is impossible. In the `presentedOccurrence` case, equality is reflexive. Therefore every survivor equals `presentedOccurrence`.

The exported `rootExactResult` conjoins the positive decision, the negative decision and the unique-survivor theorem. Lean's axiom audit prints an empty axiom dependency list for `presentationIsOccurrence`, `uniqueSurvivor` and `rootExactResult`. "No axioms" here has a precise software meaning: no axiom, sorry, classical choice declaration or

other extra axiom is imported by those theorems. Lean's kernel and dependent type theory remain the proof-checking metalanguage; they are not registered as SFT model premises.

### 3.4 Scope of uniqueness

The uniqueness theorem quantifies over `ChallengeClass`, the exact two-member registered operational grammar. It therefore proves one survivor in that complete encoded grammar. The scientific claim that this partition covers the declared operational boundary is represented by the constructor exhaustiveness and by the source registration to which the code is bound. The theorem should not be paraphrased as quantification over every grammar that any critic might later invent without first giving a structure-preserving translation into the registered boundary.

## 4. Proof-bearing whole-claim gates

`SFTValidation/Gates.lean` defines a `ClaimGate` with twelve Boolean fields:

1. identity bound;
2. source artifacts bound;
3. dependency closed;
4. candidate enumeration complete;
5. decision coverage complete;
6. exactly one survivor;
7. minimality passed;
8. named-shape uniqueness passed;
9. structural controls passed;
10. empirical boundary passed;
11. certificate bound;
12. receipt admitted.

`ClaimGate.Accepted gate` is a proposition containing a proof that every field equals `true`. `ClaimGate.certify?` checks the fields in sequence. Only the all-true path returns `some` proof certificate; every false path returns `none`. Downstream theorems extract source binding, dependency closure, complete candidate and decision coverage and exactly-one-survivor facts from any certificate.

This design prevents a successful runtime label from floating free of the logical gates. The runtime code must produce a certificate inhabiting the accepted type for each claim. The certificate is erased or compiled as ordinary Lean computation as appropriate, but its construction was checked by the kernel.

## 5. Whole-model executable audit

### 5.1 Inputs

The verifier binds the exact census and execution-manifest file hashes recorded in the final report:

- census: `sha256:f3f540f77a9b677bf7ddb9f5c6ea1ea41f08e57bb07163e25f385fd4d1dcde71;`
- execution manifest:  
`sha256:517fd288f4484f43fdc0343b17fcabd06ea9e12c977a4512d4a891dd038fcea41.`

It parses each ordered claim and its execution entry, resolves the package, and reconstructs the gates. It requires exact claim, branch, grammar and dependency identities. It checks candidate identifiers for uniqueness, expected and actual cardinality equality, a one-to-one decision map and exactly one `survives = true` row. It validates closure scope and boundary, minimality and named-form uniqueness, all four required controls, registered empirical extensions, certificate hashes and the current admitted receipt.

The verifier also preserves historical lineages without counting them as current authority: 200 prior receipt lineages and 200 prior certificate-source lineages were encountered. It checked 80 custom decision artifacts. Two uncensused nonmodel package directories were reported rather than silently absorbed: an empty future package and a rejected adverse package whose rejected receipt remains preserved. Neither is counted among the 2,777 accepted claims.

## 5.2 Complete result

| Quantity                     | Verified result |
|------------------------------|-----------------|
| Claims in ordered census     | 2,777           |
| Accepted proof-bearing gates | 2,777           |
| Source-bound claims          | 2,777           |
| Candidates                   | 898,902         |
| Decisions                    | 898,902         |
| Controls                     | 11,108          |
| Branches                     | 17              |
| Issues                       | 0               |

The exact branch distribution is:

| Branch                          | Claims |
|---------------------------------|--------|
| astronomy_cosmology             | 77     |
| biology                         | 82     |
| chemistry                       | 281    |
| computation                     | 375    |
| consciousness_cognitive_science | 77     |
| cross_branch_synthesis          | 12     |
| earth_environment               | 75     |
| engineering_translation         | 80     |
| foundation                      | 16     |
| information_science             | 262    |
| materials                       | 289    |
| mathematics                     | 341    |
| medicine                        | 76     |
| physics                         | 368    |
| quantum_computation             | 290    |
| social_collective               | 4      |
| social_collective_systems       | 72     |

The totals reconcile exactly: every ordered claim reached an accepted certificate, every claim passed the source-binding gate, candidate and decision totals match, and the issue list is empty.

## 5.3 Complete claim-inventory boundary

The standalone conceptual paper does not duplicate 2,777 full claim records. The scientific audit layer is the successor ToE matrix and the per-paper evidence maps. The machine archive remains the ordered census and the claim packages. Completeness is tested by requiring every census identifier, authoritative statement, candidate and decision cardinality, survivor, control, certificate and receipt binding to appear in the checked graph. The branch table above is the complete ownership partition for this report.

## 6. Preserved fail-closed event and correction

The first whole-model execution did not pass. It halted on six registered source-capture hashes. Investigation showed that the working-tree copies had been normalised to LF line endings while the registered source identities referred to the exact archived bytes. This was a custody mismatch even though the scientific text was not a new target.



The response was not to weaken the verifier, change the expected hashes or relabel the halt. The six exact registered byte sequences were restored from the preserved archive reference. Repository attributes were added so those specific source captures are treated as byte-preserved rather than text-normalised. A separate sweep checked all 385 registered external artifact bindings and found no remaining mismatch. The Lean verifier was then rerun without changing its acceptance logic and returned PASS.

This event is evidence about the verifier's sensitivity. It shows that a source identity mismatch blocks acceptance even when a human might regard line endings as semantically harmless. It is not evidence that the original mismatch was a scientific falsification, and the final report does not count it as one. Both the halted state and the correction rationale remain documented.

## 7. Interpretation: how the result bears on SFT

The result favors SFT in five bounded ways.

First, the root is not merely described in prose. An independent theorem prover accepts a faithful finite encoding and verifies the unique survivor constructively without extra declared axioms. Second, the current model surface is exhaustive relative to the ordered census: all 2,777 entries were traversed, not a favourable sample. Third, exactly-one-survivor is enforced at both the artifact level and the proof-bearing gate level. Fourth, source, dependency, certificate and receipt identities remain coupled across all branches. Fifth, the mismatch halt demonstrates that the layer can reject the live workspace rather than being engineered only to emit success.

These are substantial supports for internal coherence, implementation independence, provenance custody and reproducibility. They answer the concern that a large cross-disciplinary model might contain omitted branches, duplicated decisions, unmatched receipts or a root theorem that cannot be rendered precisely enough for a proof assistant.

The result is not an empirical universal confirmation. A formally coherent model can still make an empirically adverse prediction. The source gate verifies the registered custody and protocol artifacts; it does not recreate every instrument, patient cohort, astronomical survey or chemical measurement inside Lean. Likewise, the root uniqueness proof is relative to the exact operational grammar. Comparative superiority requires rival models, common observables and discriminating tests. The paper therefore reports formal validation as a strong but typed kind of evidence.

## 8. Reproduction

From generated/lean4\_validation/, with the pinned toolchain installed locally:

```
ELAN_HOME="$PWD/.elan" PATH="$PWD/.elan/bin:$PATH" lake build
ELAN_HOME="$PWD/.elan" PATH="$PWD/.elan/bin:$PATH" .lake/build/bin/sft-verify ../../reports/whole_model_validation.js
```

The checked-in ./run\_validation.sh performs the same build-and-run route. A successful run must exit zero and reproduce a report whose scientific counts, input hashes and issue-free status match this paper. The report file's byte hash can differ only if non-scientific serialization metadata changes; in this version the exact expected identity is sha256:6b6a5a9a9a0b74687a6f97fb3b4fcc7455968e35c82395290837cda60d7501cf.

## Verification-source identities

| Source                                                 | SHA-256                                                                 |
|--------------------------------------------------------|-------------------------------------------------------------------------|
| generated/lean4_validation/lean-toolchain              | sha256:90f5a018ecbd1ad55afee82b7677bf492958f1786fcbf76c1fa145742f53d432 |
| generated/lean4_validation/lakefile.toml               | sha256:aefcc07b064480f8aa1b130e5167c911aff5694ebc7a021052d9942a04c41fee |
| generated/lean4_validation/Main.lean                   | sha256:d116cbaf7db3d784db90b67a8b42611ffd57c4bf75bdf7160701dc068ee05577 |
| generated/lean4_validation/SFTValidation.lean          | sha256:ceb0357e40a0e5bdd229eafa34bf1083759ae2fd8ace1668daacf18c82d0e683 |
| generated/lean4_validation/SFTValidation/Root.lean     | sha256:937a9cbf0053f36117171dba65577034641e3165562822784a4013178463132f |
| generated/lean4_validation/SFTValidation/Gates.lean    | sha256:74c956a67786bb7a846cf6ee58616fcfb3ccbc00b719d2e7f34622d2a80691da |
| generated/lean4_validation/SFTValidation/JsonUtil.lean | sha256:d648cf8f7c4ae386e1b170764fc8f93970bb91a9d75d89143c6fef812ba30f06 |
| generated/lean4_validation/SFTValidation/Verifier.lean | sha256:ce7b77b3b49af982aecc2a753905b9afa0a80762e3434a78cd6f677f33a4db0b |
| generated/lean4_validation/source_binding_probe.py     | sha256:f36264cbe615657f2d146097b54e9f817f0b52f67f4820d899b35566e0b25662 |
| generated/lean4_validation/run_validation.sh           | sha256:8d658cbb7f90dcb824d789bdb9661e95ed0e09ccd4b55cd378110d8b797e28ac |

## Open science, criticism and admission

The verification sources, paper, evidence map, report and reproduction commands are open for inspection. Criticism does not require credentials or institutional permission. Scientific admission remains separate: a proposed correction must identify the exact theorem, grammar, artifact, source, chronology or empirical boundary it challenges and must pass the shared versioned protocol. Lean acceptance, reputation, consensus and application performance cannot substitute for that route.

Maria Smith retains authorship and sole publication authority. Ernos Labs denotes conformance to the open evidence constitution, not authority over criticism. Paper and documentation are CC BY 4.0; repository verification code is Apache-2.0. Contributions and machine assistance do not transfer authorship or publication authority.

## 9. Falsification and review conditions

This verification result fails for the bound repository state if any of the following is demonstrated:

- `Root.lean` admits an extra root constructor, relies on an undeclared axiom or fails to prove uniqueness;
- the root code is not a structure-preserving encoding of the registered two-class grammar;
- a current census claim is omitted or counted twice;
- candidate cardinality, decision coverage or survivor count is accepted when inconsistent;
- a required control, empirical boundary, certificate or admitted receipt can be false while a proof-bearing certificate is returned;
- a source binding can drift without causing failure;
- the reported input hashes do not identify the audited census and execution manifest;
- the pinned toolchain cannot reproduce the build and pass result; or
- the protected engine or authority was modified to make the Lean layer succeed.

A critique of the scientific model should additionally target the registered grammar, a particular derivation, an empirical protocol, an adverse observation or a discriminating prediction. A generic statement that Lean proves only formal encodings is correct but incomplete here, because this project explicitly separates native propositions from artifact checks and binds both to the repository's evidence structure.

## 10. Limitations and next formalisation frontier

The present version natively proves the operational root and generic gate implications. It does not translate every domain statement into a bespoke Lean proposition, nor should its artifact checks be described as if it did. Formal closure, implementation validation and empirical validation remain distinct, and the empirical frontier stays open wherever a claim's current record is adverse, unavailable or unresolved. A future version may add native formalisations for selected mathematical and physical claims, along with equivalence theorems between those propositions and their registered JSON representations. Such extensions must remain versioned and cannot retroactively change this paper's result.

The read-only Python source bridge is a practical boundary rather than a hidden proof step. Eliminating it would require translating each registered source factory and transport rule into Lean or exporting a separately authenticated language-neutral registry. The current bridge is fail-closed, its role is disclosed, and its output is included in the claim gate.

The verification proves the integrity of the current census, not permanent closure against lawful discovery. A new claim changes the census and manifest hashes and requires a new report and paper version. This is a feature of the versioned scientific model: completeness is always exact to a declared dated surface.

## 11. Conclusion

Lean 4 independently verifies the exact two-class SFT operational root and proves that `presentedOccurrence` is its unique survivor. The same compiled project constructs proof-bearing acceptance certificates for all 2,777 current claims after checking 898,902 candidates, 898,902 decisions, 11,108 controls and the complete seventeen-branch evidence graph. The final report is PASS with no issue, while the preserved source-hash halt demonstrates fail-closed behaviour. Adverse and unresolved scientific records remain evidence-bound rather than reclassified; empirical testing, criticism and lawful versioned extension remain open.

The appropriate conclusion is strong and bounded: the complete current SFT model is formally coherent and machine-consistent under this independent Lean verification architecture, with intact source, dependency, survivor, certificate and receipt custody. Empirical truth and comparative physical adequacy remain questions for the model's registered experiments and falsification programme.

## Data and code availability

All verification sources, the pinned toolchain declaration, the machine report, the publication evidence map and the local release manifest are included in the repository. The model and historical evidence remain in their original locations. No external publication action was performed while preparing this candidate.

## References

1. de Moura, Leonardo, and Sebastian Ullrich. "The Lean 4 Theorem Prover and Programming Language." In *Automated Deduction - CADE 28*, Lecture Notes in Computer Science 12699, 625-635. Springer, 2021. DOI: [10.1007/978-3-030-79876-5\\_37](https://doi.org/10.1007/978-3-030-79876-5_37).
2. Lean FRO. *The Lean Language Reference*. Current reference documentation used with the pinned Lean 4.32.0 toolchain: [lean-lang.org/doc/reference/latest](https://lean-lang.org/doc/reference/latest).
3. Smith, Maria. *There Is No Nothing: A Premise-Free Operational Foundation and an Open Verification Platform for Smithian Fold Theory*. Ernos Labs Methods Paper 00, versioned publication series, 2026.
4. Smith, Maria. *The Smithian Fold Theory V3 Theory of Everything*. Exhaustive preliminary monograph and machine audit archive, versioned publication series, 2026.
5. Ernos Labs. `generated/lean4_validation/README.md`, `SFTValidation/Root.lean`, `SFTValidation/Gates.lean`, `SFTValidation/Verifier.lean` and `reports/whole_model_validation.json`, repository snapshot bound by the hashes in this paper.

## Suggested citation

Smith, Maria (2026). *Independent Lean 4 Verification of the Complete Smithian Fold Theory Model: A Kernel-Checked Operational Root, Proof-Bearing Acceptance Gates and Fail-Closed Audit of All Current Branches*. Ernos Labs, version 1.0.0. DOI pending author-confirmed deposit.

## Exact current claim-record preservation addendum

The records below are reproduced byte-for-byte from the current census and claim packages because literal-aware house-style editing must never alter an authoritative statement or receipt identity.

### Premise-free operational root theorem

**Claim ID:** SFT-ROOT-THERE-IS-NO-NOTHING

**Exact statement:** No admissible operational statement, denial, record, proof object or derivational object is nothing: absence presents no counterexample, while every presented counterexample is an occurrence.

**Dependency route:** Root; no preceding dependency

**Candidate generator:** Partition every purported operational counterexample by whether it emits an identifiable presentation at the proof boundary.

**Grammar boundary:** All operational statements, denials, records and proof challenges, independent of their internal length or notation.

**Complete census:** 2 candidates; 1 unique survivor; 4 controls

**Survivor:** presented-occurrence

**Exact result:** presented-occurrence is the unique surviving operational class

**Closure:** depth\_independent; minimality True; named-shape uniqueness True

**External status:** independently\_replicated

**Engine receipt:** sha256:711864171e4d3a2f2734f0c2890965bcd81a0228349538751a3c80699c27d669 at receipts/engine/model\_admitted/SFT-ROOT-THERE-IS-NO-NOTHING-711864171e4d3a2f.json

| Control           | Passed | Expected                                                           | Observed                                                     |
|-------------------|--------|--------------------------------------------------------------------|--------------------------------------------------------------|
| false_premise     | True   | reject because no counterexample identity or trace exists          | unpresented-absence is not admissible as a counterexample    |
| tampered_source   | True   | reject source drift before enumeration                             | the altered identity differs from the registered identity    |
| tampered_artifact | True   | reject because forcing no longer has a unique survivor             | the tampered survivor set contains both named classes        |
| boundary          | True   | refuse the broadened claim because it supplies no checkable object | the theorem remains scoped to admissible operational objects |